

mb-free__transport-rendezvous-512

An AgentMPI run analysis

Abstract

This is the second cell of the rendezvous column in a ten-cell transport sweep — two modes crossed with five payload sizes, all at $p = 8$ and all agent-free. Eight ranks ring-exchange a 512-word payload under `Mode.RENDEZVOUS`, so only an envelope is delivered and the content stays in the fabric behind a digest. It completed in 0.02s over 34 events with zero admissions and a context high water of zero on every rank, deferring 5,392 tokens. Its eager twin admitted 674 tokens per receiver and also completed, so nothing here demonstrates necessity. What this cell contributes is the second point on the deferral line: 5,392 is 8×674 , one send per rank, and it is exactly $4.01 \times$ the 128-word cell's 1,344 for $4 \times$ the payload. The deferred figure is linear in payload and equal to `tokens sent`, which is what it should be when every message is rendezvous — and which is why the 10,784 recorded for this cell in `results/microbench/free-transport.json`, exactly twice the log-derived value, is a defect rather than a rounding difference.

1 What this run is

property	value
run	mb-free__transport-rendezvous-512
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	none $\times$ 8
events	34
wall time	0.02s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	0 eager, 8 rendezvous
tokens sent	5,392	5,392 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	\$0.0000	0.0000 \$/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

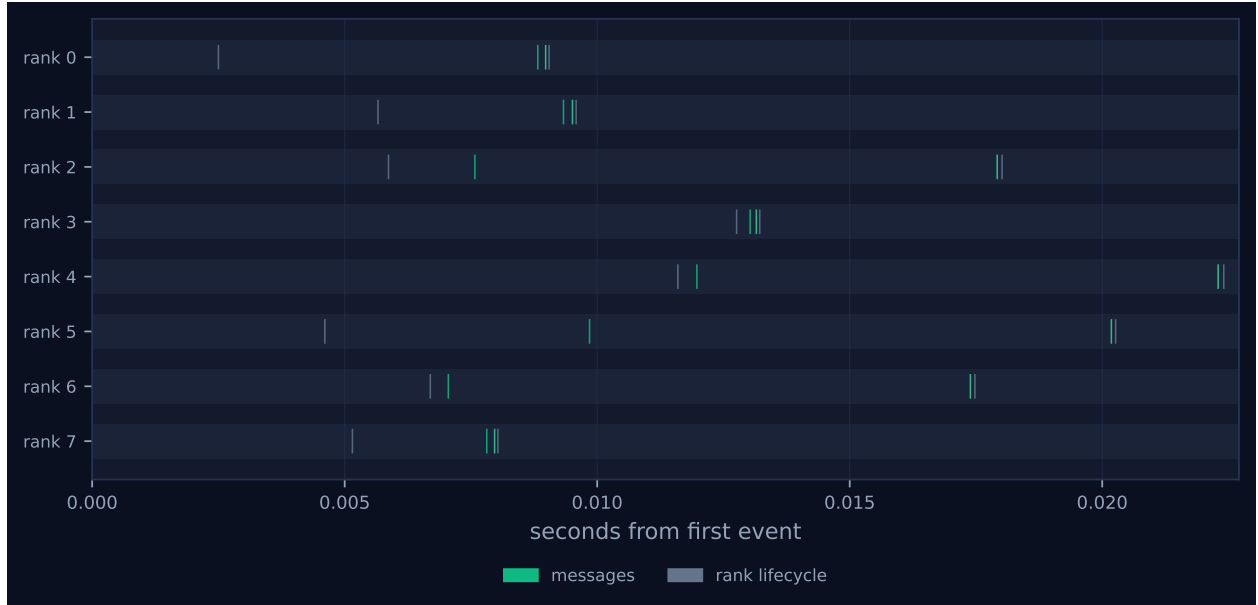


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

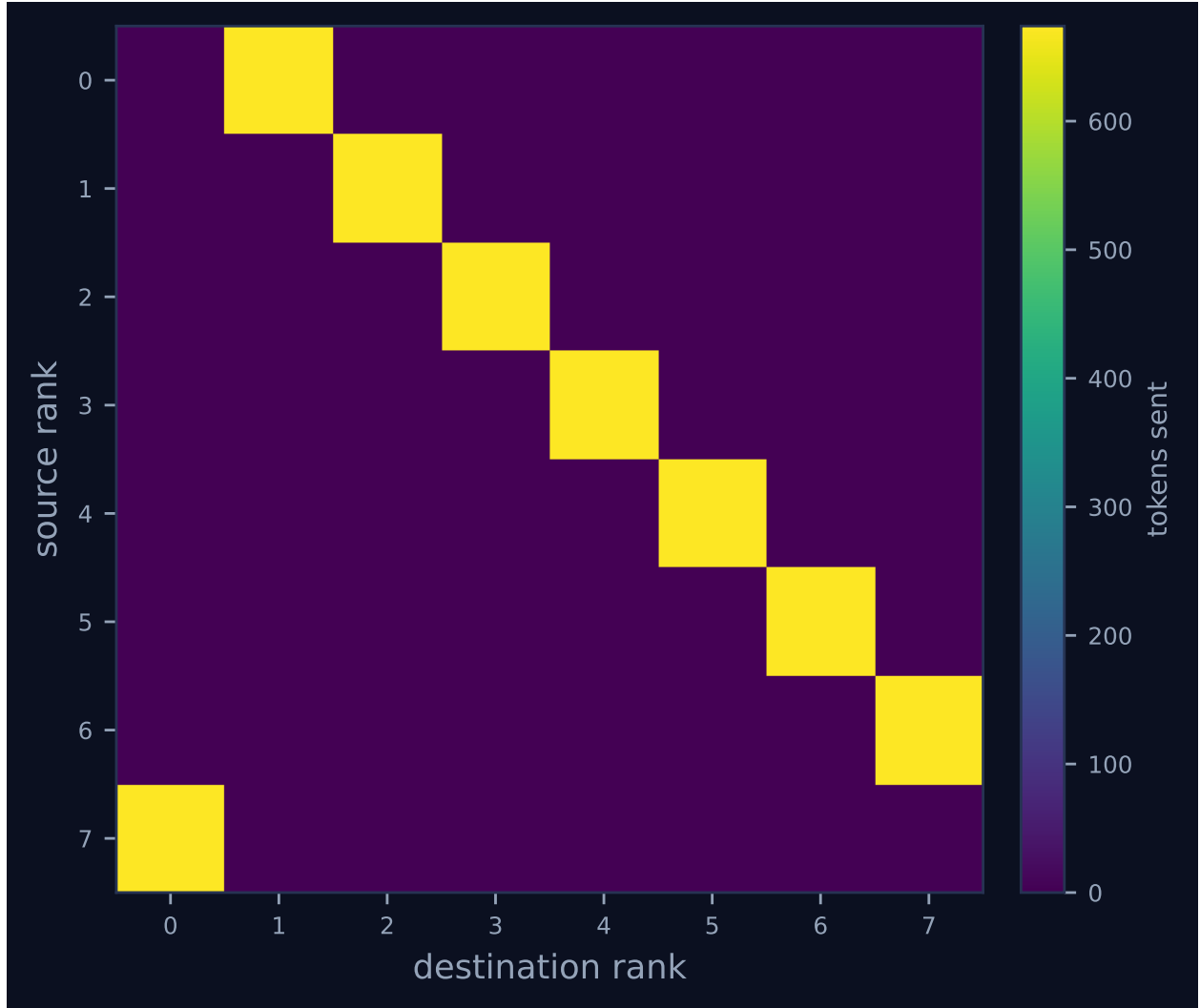


Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	674	0.0	finalized
1	0.00	0.0	0	0	1	1	674	0.0	finalized
2	0.00	0.0	0	0	1	1	674	0.0	finalized
3	0.00	0.0	0	0	1	1	674	0.0	finalized
4	0.00	0.0	0	0	1	1	674	0.0	finalized
5	0.00	0.0	0	0	1	1	674	0.0	finalized
6	0.00	0.0	0	0	1	1	674	0.0	finalized
7	0.00	0.0	0	0	1	1	674	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 is eight lanes of instantaneous marks: two grey lifecycle diamonds and two green message ticks per rank, no bars. The missing extent is a property of the benchmark rather than a finding — the executor is **none** on all eight ranks, there were zero agent calls, and no **task.*** event was emitted — so the headline table’s “idle fraction 100.0%” and “achieved parallelism 0.00×” mean that no rank held a task, not that ranks waited. Ranks arrive over a 10 ms window inside a 23 ms span, and the ring completes with a median receive latency of 7.5 ms and a maximum of 12 ms — figures that reflect rank arrival order and contention on the single SQLite writer, since every send inserts a row and every receive claims one inside an exclusive write transaction. Nobody blocks; there is no **transport.credit_stall** in the log, and there could not be, because the credit mechanism guards eager traffic only and this run has none. Figure 2 shows the ring as one off-diagonal band of 674 tokens per edge.

The rank health table is where the mode shows: 0% occupancy on all eight ranks, against 2% for **mb-free__transport-eager-512**. The two runs are otherwise near-identical — the same 2,560-byte payload, the same single deduplicated blob, eight envelopes, 34 events. Every **msg.recv** here reports **admitted: false** with **admitted_tokens: 0** against **tokens: 674**, and every **rank.finalize** reports zero admissions and zero items. The viewer’s stats row reads **TOKENS DEFERRED 5.4k**, which is the corrected, log-derived total rather than the number in the results file.

7 Interpretation

By construction: the world size, the ring, one send and one receive per rank, and the mode — forced, since **rank.init** records **eager_limit: 1000000000** and a 674-token payload would otherwise have travelled eager under the shipped default of 2,048 tokens. The non-admission is likewise a

default and not a receiver-side decision: the benchmark calls `recv` with no `admit` argument, and `Communicator._deliver` resolves `admit = None` to `mode != rendezvous`. The sender’s choice of transport determined the receiver’s context policy, which is the semantics `constants.Mode` claims and this run confirms without the harness doing anything.

The arithmetic is the substance. The payload is 512 repetitions of "word ", 2,560 characters, priced by the default estimator at $\lceil 2560/3.8 \rceil = 674$ tokens. With eight ranks and one send each the log-derived deferral in `metrics.json` is $8 \times 674 = 5,392$, identical to `tokens sent`, since 100% of the traffic is rendezvous. Against the 128-word cell’s 1,344 that is $4.01\times$ for $4\times$ the payload; against the 2,048-word cell’s 21,560 it is $0.250\times$ for a quarter of the payload. There is no per-message overhead in the deferral figure, just as there is none in eager’s context charge, and the whole rendezvous column is 8×1.3158 tokens per payload word. The results file, however, records 10,784 — and the per-rank `rank.finalize` events say why: each rank reports `tokens_deferred: 1348` against `tokens_sent: 674`, deferring twice what it sent. Before commit 0fea51d2, `RankCost.tokens_deferred` accumulated on both rendezvous sends and non-admitted receives, so every rank charged its own send and its predecessor’s arrival to the same counter. The commit split the two fields into `tokens_deferred` (send-side, rendezvous only, a subset of `tokens_sent` by construction) and `tokens_unadmitted` (receive-side, transport-agnostic), and under those definitions this run would report 5,392 of each. `cost.summarise` derives from the log and was always correct, which is why `metrics.json` and the viewer already show 5,392; only the archived results carry the doubled figure. The fix’s own summary holds that the paper’s transport table is unaffected because the microbench admits its receives — true of the eager rows, which correctly read zero, and false of the rendezvous rows, because nothing in `bench_transport` passes `admit` and rendezvous defaults to `False`. All five rendezvous entries in that table are exactly $2\times$ the send-side deferral.

Placed in the design, this cell is a control rather than a demonstration. Its eager twin used 2.1% of a receiver’s 32,768-token budget and 16.5% of the 4,096-token credit, both far from binding, and it delivered the text. This run delivered a handle that no receiver opened. Both finished in about 20–30 ms. Rendezvous is a reasonable choice at this size and it is not a required one, and the sweep’s actual argument lives two cells up, where the payload’s 10,779 tokens exceed the eager credit and the eager arm produces 18 events and no messages at all. The generated finding list is empty, correctly.

8 Threats to this reading

The zero-context result costs the receiver the content. This run never called `fetch`, so “zero admissions” means nothing was read, and a harness that materialised the payload with `admit=True` would consume the same 674 tokens the eager twin did. The transport sweep contains no fetch at any size; the ping-pong in `experiments/microbench.py`, which does `recv(admit=False)` followed by `comm.fetch(msg, admit=False)`, is where the full rendezvous pattern is exercised. Second, with `executor: none` and zero agent calls, every context number here is an entry in `ContextAccount` rather than a measured prompt, and the run says nothing about agent behaviour. Third, the token counts are estimates (`tokenizer_exact: false`); the clean 5/3.8 ratio that makes the deferral line so straight is an artefact of a character heuristic applied to one repeated word, and a real BPE tokeniser would price this payload near 512 tokens, scaling every figure in this document by about 0.76. Fourth, the wall time is a single sample of 23 ms against a 10 ms start-up spread and SQLite writes, so the fact that this cell is nominally faster than its eager twin means nothing. Finally, the

doubled deferral is diagnosed from per-rank arithmetic and from the pre-fix source, not observed directly — the code that wrote these traces is no longer in the tree, and the reasoning depends on the claim that a rank whose only actions were one 674-token rendezvous send and one non-admitted 674-token receive could not otherwise have reached 1,348.